



BITS Pilani
Pilani Campus

API-driven Cloud Native Solutions

Dr. Shreyas Rao
Associate Prof. (Off Campus), CSIS, BITS-Pilani



CC ZG506 / SE ZG506

Lecture No. 9

Agenda

AI and Cognitive Services

Part-A: Basic APIs for AI applications

- What are Language Models?
- LM Types
 - Large Language Model (LLM)
 - Small Language Model (SLM)
- LLMOps
- Demo / Hands-on on SLMs and LLMOps

Language Models

- Language models are a type of artificial intelligence (AI) model designed to understand and generate content based on human language.
- Includes NLU and NLG components.
- They are trained on large datasets of text to learn the patterns, syntax, semantics, and context within language



Large Language Models (LLM)

- Large language models (LLMs) are deep learning algorithms that can recognize, extract, summarize, predict, and generate text based on knowledge gained during training on **very large datasets**.
- They are trained on large sets of data, comprising of books, articles, and web pages.
- These models are designed to understand the **context and meaning of text** and can **generate text that is grammatically correct and semantically relevant**.
- They can be trained on a wide range of tasks, including language translation, summarization, question answering, and text completion

Ref: "A Beginners Guide to Large Language Models" by NVidia



LLM Development Timelines

1950s–1990s

Initial attempts are made to map hard rules around languages and follow logical steps to accomplish tasks like translating a sentence from one language to another.

While this works sometimes, strictly defined rules only work for concrete, well-defined tasks that the system has knowledge about.

1990s

Language models begin evolving into statistical models and language patterns start being analyzed, but larger-scale projects are limited by computing power.

2000s

Advancements in machine learning increase the complexity of language models, and the wide adoption of the internet sees an enormous increase in available training data.

2012

Advancements in deep learning architectures and larger data sets lead to the development of GPT (Generative Pre-trained Transformer).

2018

Google introduces BERT (Bidirectional Encoder Representations from Transformers), which is a big leap in architecture and paves the way for future large language models.

2020

OpenAI releases GPT-3, which becomes the largest model at 175B parameters and sets a new performance benchmark for language-related tasks.

2022

ChatGPT is launched, which turns GPT-3 and similar models into a service that is widely accessible to users through a web interface and kicks off a huge increase in public awareness of LLMs and generative AI.

2023

Open source LLMs begin showing increasingly impressive results with releases such as Dolly 2.0, LLaMA, Alpaca and Vicuna.

GPT-4 is also released, setting a new benchmark for both parameter size and performance.

Large Language Models (LLM)

What is “Large”??

Table 1. Comparison of MT-NLG and GPT-3

Large Language Model	Number of parameters	Number of tokens in the training data
NVIDIA Model: Megatron-Turing Natural Language Generation Model (MT-NLG)	530 billion	270 billion
OpenAI Model: GPT-3 Davinci Model	175 billion	499 billion

Ref: “A Beginners Guide to Large Language Models” by NVidia



What are Model Parameters?

- Parameters are the **individual variables within a neural network** that are learned from training data.
- In language models, parameters help the model to interpret relationships between words, phrases, and concepts.
- These parameters are adjusted during training to minimize prediction error, allowing the model to generate coherent, contextually accurate text.
- With 175 Billion parameters, GPT-3 can **understand subtle relationships within language**, allowing it to generate detailed, contextually relevant responses to a wide range of prompts.

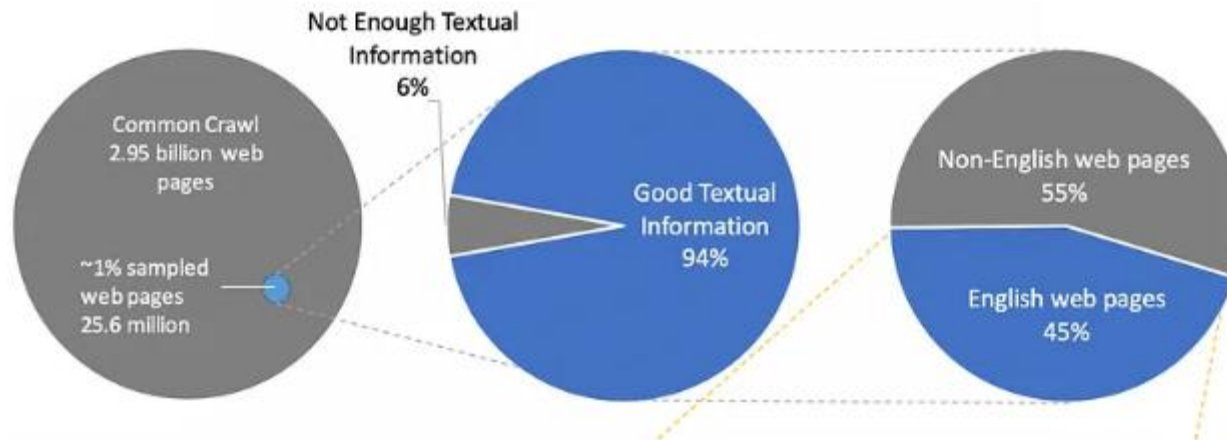
Ref: "A Beginners Guide to Large Language Models" by NVidia



Dataset of Web Resources used for building LLMs

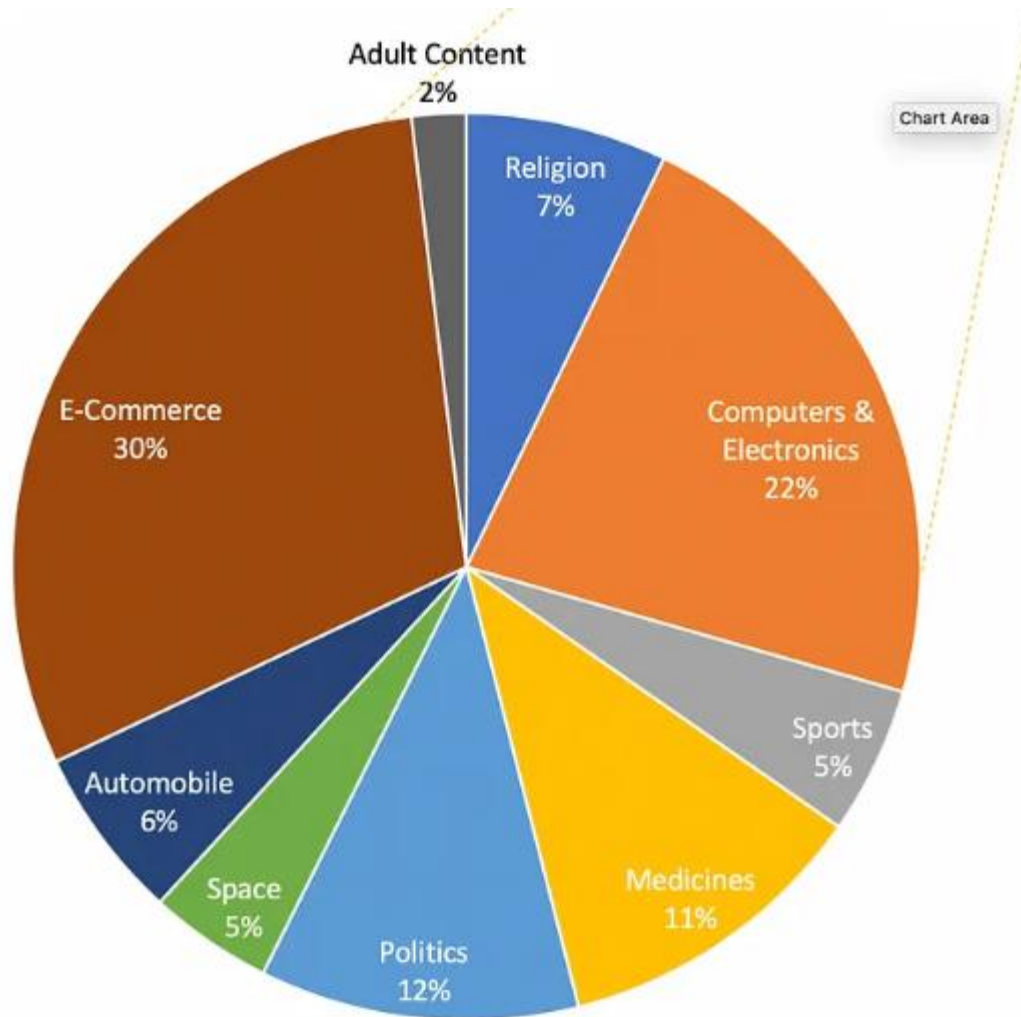
- Common Crawl
- Over 250 billion pages spanning 17 years.
- Free and open corpus since 2007.
- Cited in over 10,000 research papers.
- 3-5 billion new pages added each month.
- URL - <https://commoncrawl.org/>

Common Crawl



Ref: <https://towardsdatascience.com/categorizing-world-wide-web-c130abd9b717>

Common Crawl



What are Tokens?

- In LLMs, tokens are the “smallest units” of text that the model processes
- Tokens are the minimal, discrete pieces of input text that the model processes. For example, in the phrase "I love programming," tokens could be individual words ("I," "love," "programming") or even parts of words, depending on how the model splits them.
- Sub-word tokenization: Most modern LLMs, like GPT and BERT, use subword tokenization method. For instance, “unbelievable” might be tokenized as ["un", "believ", "able"]

Token impact on Cost of LLM

- The tokenization strategy affects the length of input the model can process at once, which also impacts computational cost.
- For instance, if a model can process 4,096 tokens per request, this includes both input and output tokens in that limit.
- Prompt: “What is the capital of India?”
- Response: Capital of India is New Delhi
- In this example, if we consider one word as one token, then number of tokens consumed is 12.

Popular LLMs

- GPT-4 (OpenAI)
- LLaMA 3 (Meta)
- Claude 3 (Anthropic)
- Mistral 7B
- PaLM 2 [Google - primarily text]
- Gemini [Google - Multimodal]
- Grok (xAI) [Elon Musk]
- ERNIE 4.0 (Baidu) [Chinese]
- Falcon (Technology Innovation Institute)
- DeepSeek R1



Small Language Models (SLM)

Small Language Models (SLMs)

1. LLMs operate on Billions or Trillions of parameters – require huge computational power.
2. SLMs operate on a more **modest scale**, typically ranging from **thousands to a few million parameters**.
3. This reduced scale brings about lower computational requirements, rendering smaller language models more accessible and feasible for organizations or researchers who may lack the resources to manage the substantial computational load demanded by larger models.

<https://www.leewayhertz.com/small-language-models/#Distinguishing-SLMs-from-Large-Language-Model>

SLM Advantages



LeewayHertz

1. Ex: "Phi-3.5-mini-instruct" model is fine tuned for coding related tasks
2. Efficient resource usage – CPU / Hardware constraints

<https://www.leewayhertz.com/small-language-models/#Distinguishing-SLMs-from-Large-Language-Model>

Differences between LLMs and SLMs

LLMs are general purpose [all tasks such as NLP, CV etc.], whereas SLMs are specific for some tasks only [Text generation or machine translation etc.]

<https://www.leewayhertz.com/small-language-models/#What-are-the-strategic-advantages-of-SLMs-for-enterprises>

Popular SLM Models

- DistilBERT [based on Google BERT LLM Model] – Fine tuned for Text Classification & Q&A
- TinyBERT [Even smaller than DistilBERT] – Fine tuned for Sentiment Analysis, Q&A, Text Summarization
- GPT-NeoX-20B Lite
- TinyGPT
- Phi-3.5-mini-instruct [Microsoft] – fine tuned for text summarization, Q&A, Dialog & Chatbots, text generation, text classification, code related questions etc.
- microsoft/Phi-4-multimodal-instruct [5.6 Billion parameters]

Parameter Ranges for Different Model Categories

Model Type	Parameter Range	Examples
Small Language Model (SLM)	1B - 10B params	Phi-3 (3.8B), Phi-4 (5.6B), Mistral-7B, Llama 2-7B
Medium-sized LLM	10B - 50B params	Llama 2-13B, GPT-3 (13B variant), Falcon-40B
Large Language Model (LLM)	50B - 175B+ params	GPT-3 (175B), Llama 2-65B, Claude 3
Giant-scale LLM	>175B+ params	GPT-4 (estimated 1T), PaLM-2, Claude 3 Opus



“microsoft/Phi-3.5-mini-instruct” SLM model

1. Create a HuggingFace Token (Free)
2. Create Login in HuggingFace
3. Go to <https://huggingface.co/settings/tokens> website, and “Create New Token”
4. Use the HuggingFace Chat Interface to interact with the model

URL - <https://huggingface.co/chat/models/microsoft/Phi-3.5-mini-instruct>

Select “Current Model” as “microsoft/Phi-3.5-mini-instruct”

SLM Models – Demo and Hands On

The following demonstrates “inferencing” using SLM models

1. Write Python code to answer code related or general knowledge questions using Microsoft Phi-3 SLM model and Hugging Face APIs
2. Write Python code to import a Word document, and implement a simple chatbot interface to answer questions based on the SLM model. Use Microsoft Phi-3 SLM model for this activity.

* Here, “context” refers to the “entire document” that is input.

<https://colab.research.google.com/drive/1QA3zRE-PkR5rwXn9C9ym2ku7WgAAer7J#scrollTo=IMCaXRjd0sBL>

SLM Hyperparameters

1. stream=True:

Enables streaming mode, meaning the model will send tokens progressively rather than waiting for the entire output to complete. This is useful for real-time applications where partial responses are beneficial, like in chat applications.

2. temperature=0.5:

Controls the randomness or creativity of the model's responses. Values close to 0 make the output more deterministic, focusing on likely words, while higher values increase diversity. A setting of 0.5 provides a balance between creativity and predictability, making responses both coherent and somewhat varied.

Ex 1: For Question Answering, we may want to keep at 0.2-0.3, since we want the answer from the corpus only. [Factual answer, not imaginative]

Ex 2: For Text generation, like "write a poem", we may keep temperature = 0.8, to allow creativity and not restrict to known vocabulary or sentences.



SLM Hyperparameters

3. max_tokens=1024:

Sets the maximum number of tokens the model will generate in the output. Tokens include words, subwords, and punctuation marks, so the output will not exceed this token limit.

4. top_p=0.7:

Controls nucleus sampling, where the model selects from the smallest group of words that together have a cumulative probability of 0.7. This constrains the output to likely options without restricting it too much, promoting coherent but creative responses. Lower values restrict output to highly likely words, while higher values increase the diversity of choices.

SLM Hyperparameters

Top_p

Sentence - "Once upon a time, there was a dragon who loved to..."

top_p=1.0 (No filtering): The model will have a large pool of tokens to choose from, resulting in varied responses. It might generate something like:

"...fly over mountains and watch the stars."

"...collect shiny objects and treasure from humans."

"...bake cookies and share them with villagers."

top_p=0.7: With top_p=0.7, the model will only consider tokens that are highly probable, resulting in responses that are creative but still coherent

"...collect shiny objects."

"...fly over mountains."

"...guard its treasure."



BITS Pilani
Pilani Campus



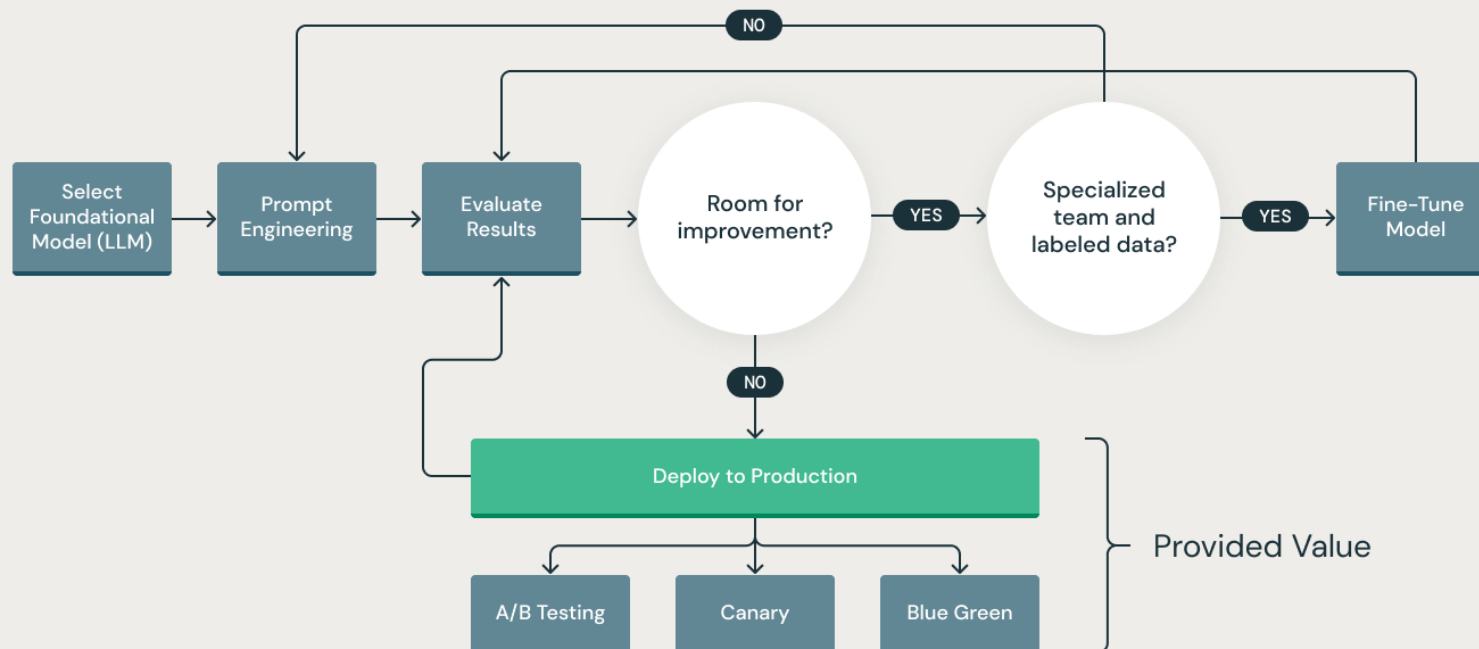
LLMOps / SLMOps

LLMOps

Large Language Model Ops (LLMOps) encompasses the practices, techniques and tools used for the operational management of large language models in production environments.

Ref: <https://www.databricks.com/glossary/llmops>

Example of development-to-production workflow for LLMs



Ref: <https://www.databricks.com/glossary/llmops>

SLMOps – Demo and Hands On

3. Implementing the earlier SLM Code using LLMOps - MLFlow tool, which is executed within Google Colab. The log metrics data is stored in JSON format

```
{"model": "microsoft/Phi-3.5-mini-instruct", "question": "what is hardware independence?", "latency": 0.25985002517700195, "question_token_count": 4, "context_token_count": 548, "response_token_count": 46, "response_length": 333, "estimated_cost": 0.0005, "response_text": " Hardware independence refers to the ability of any virtual machine to be provisioned or migrated to any physical server without compatibility issues. This means that VMs can run on different hardware configurations, simplifying disaster recovery, load balancing, and ensuring flexibility in managing VMs across various environments."}
```



Thank You!